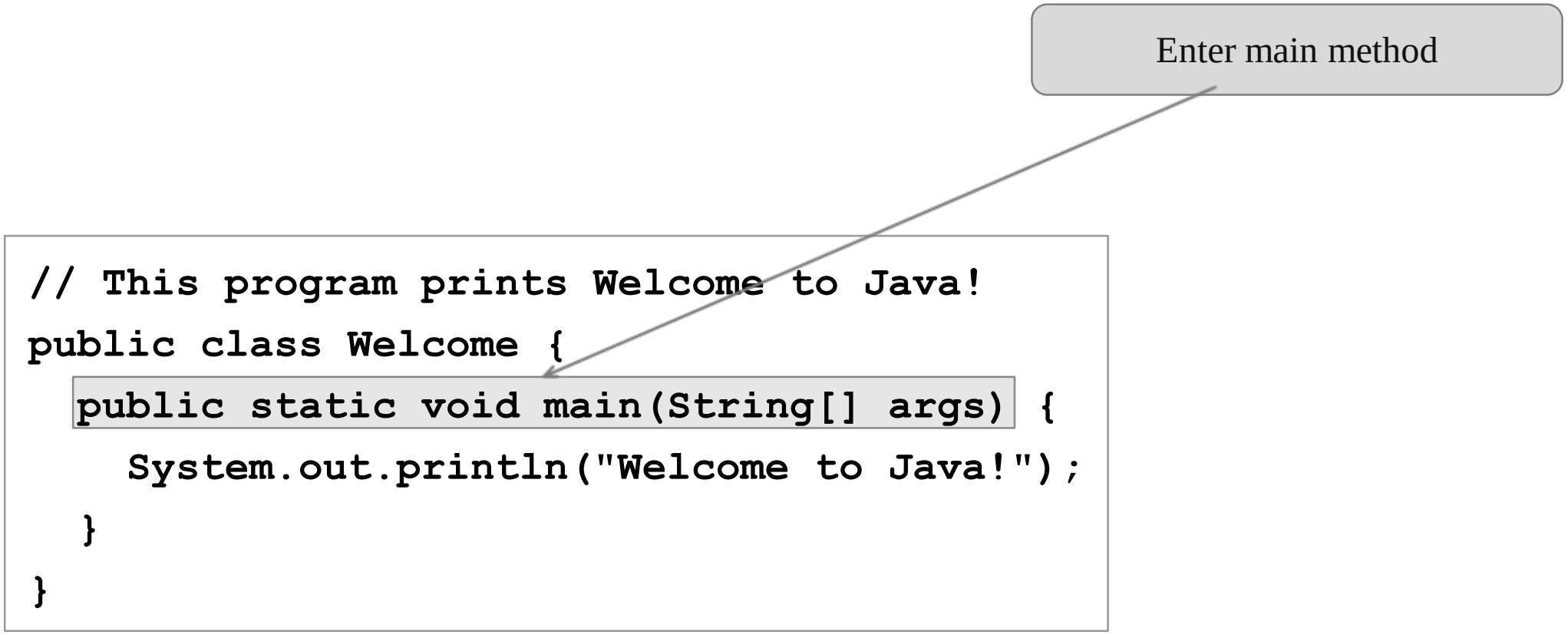
Seminar 1 · Java basics

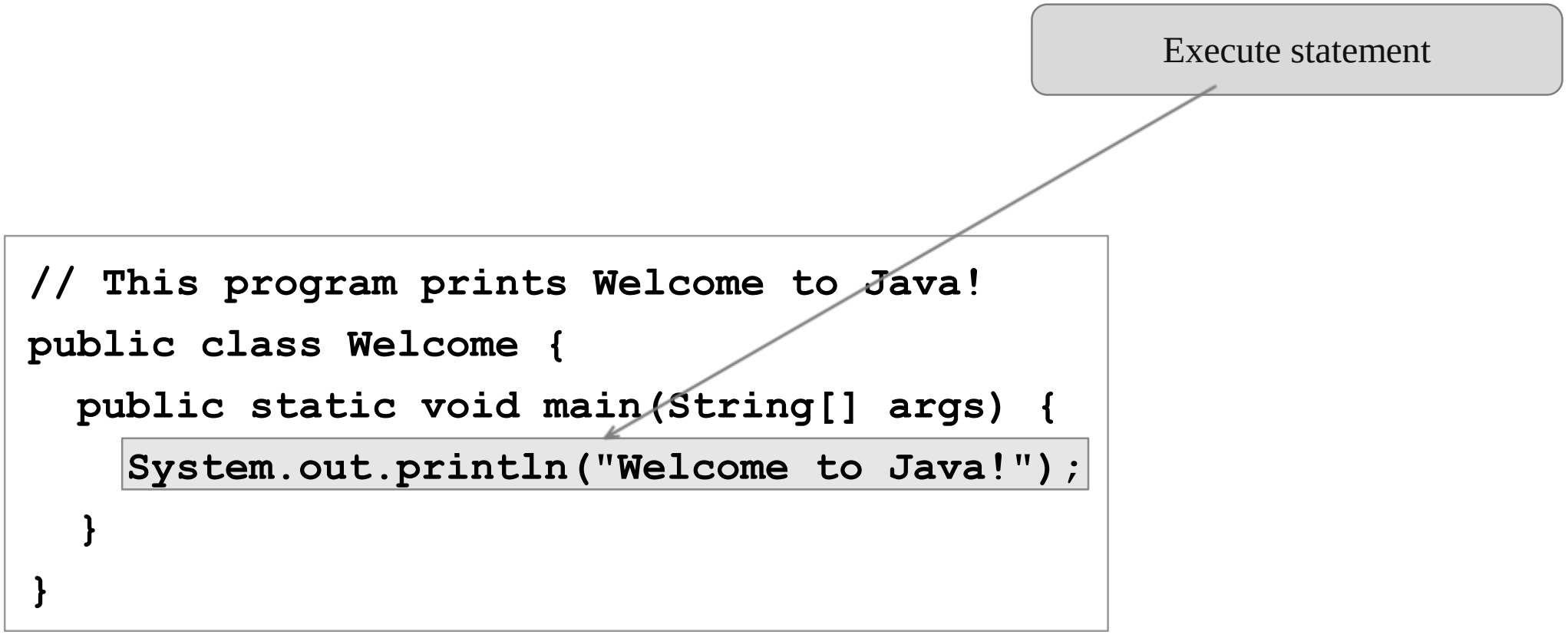
Anatomy of a Java program

Tracing Welcome.java, the language basics, and the three kinds of errors

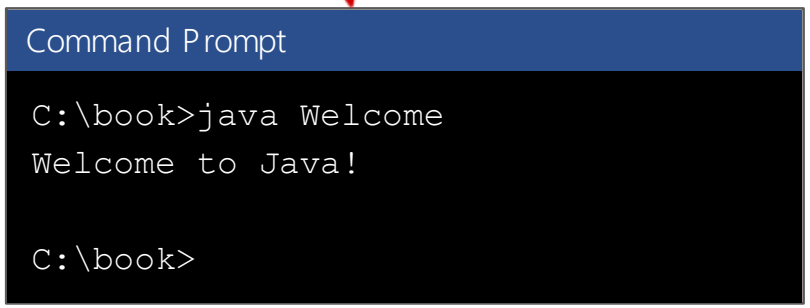
Trace a Program Execution



Trace a Program Execution



```java
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```



print a message to the console

Class Name

Every Java program must have at least one class. Each class has a name. By convention, class names start with an uppercase letter. In this example, the class name is Welcome.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Main Method

Line 2 defines the main method. In order to run a class, the class must contain a method named main. The program is executed from the main method.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

```
1  public class Welcome {
2    public static void main(String[] args) {
3      // Display message Welcome to Java! on the console
4      System.out.println("Welcome to Java!");
5    }
6  }
```

The program starts the execution from the main method.

```
1  public class Welcome {
2    public static void main(String[] args) {
3      // Display message Welcome to Java! on the console
4      System.out.println("Welcome to Java!");
5    }
6  }
```

The statement displays Welcome to Java! to the console.

Output

```
Welcome to Java!
```

```
1  public class Welcome {
2    public static void main(String[] args) {
3      // Display message Welcome to Java! on the console
4      System.out.println("Welcome to Java!");
5    }
6  }
```

The main method exits. The program is finished.

Statement

A statement represents an action or a sequence of actions. The statement System.out.println("Welcome to Java!") in the program in Listing 1.1 is a statement to display the greeting "Welcome to Java!".

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Statement Terminator

Every statement in Java ends with a semicolon (;).

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

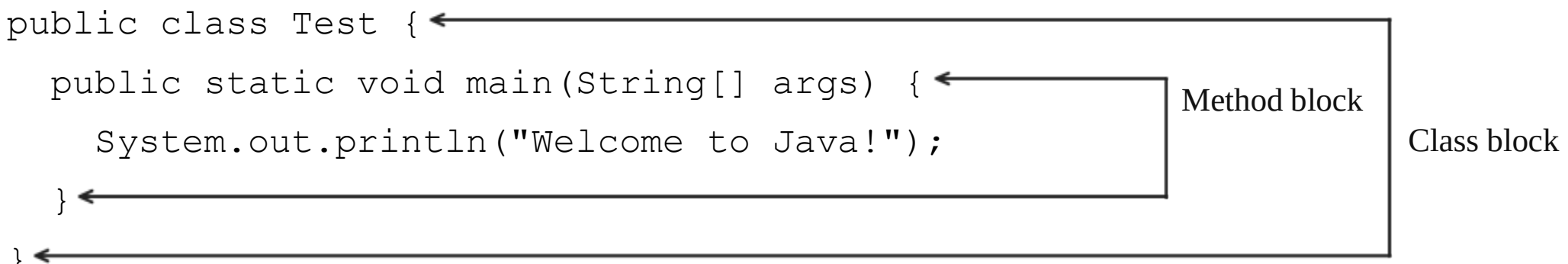
Reserved words

Reserved words or keywords are words that have a specific meaning to the compiler and cannot be used for other purposes in the program. For example, when the compiler sees the word class, it understands that the word after class is the name for the class.

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Blocks

A pair of braces in a program forms a block that groups components of a program.



Special Symbols

Character	Name	Description
{}	Opening and closing braces	Denotes a block to enclose statements.
()	Opening and closing parentheses	Used with methods.
[]	Opening and closing brackets	Denotes an array.
//	Double slashes	Precedes a comment line.
" "	Opening and closing quotation marks	Enclosing a string (i.e., sequence of characters).
;	Semicolon	Marks the end of a statement.

{ ... }

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

(...)

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

;

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

// ...

```
// This program prints Welcome to Java!
public class Welcome {
  public static void main(String[] args) {
    System.out.println("Welcome to Java!");
  }
}
```

Comments

- C-style comments (multi-lines)

```
/* this comment is so long
   that it needs two lines */
```

- Comments on a single line

```
// comment on one line
```

Appropriate Comments

Include a summary at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.

Include your name, class section, instructor, date, and a brief description at the beginning of the program.

Naming Conventions

- Choose meaningful and descriptive names.
- Class names:

– Capitalize the first letter of each word in the name. For example, the class name ComputeExpression.

Block Styles

Use end-of-line style for braces.

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

Classes and Primitive Types

Type

- Class

```
class Exam {}
```

- Primitive type

```
int, char, float
```

Instance

- Variable of type reference

```
Exam e;
e = new Exam();
```

- Variable of type primitive

```
int i;
```

Operators

- Operators follow C syntax:
  - arithmetical + - * / %
  - relational == != > < >= <=
  - bitwise (int) & | ^ << >> ~
  - assignment = += -= *= /= %= &= |= ^=
  - increment ++ --

Boolean

- Java has an explicit type (**boolean**) to represent logical values (**true**, **false**)
- Conditional constructs require boolean conditions
  - Illegal to evaluate an integer condition

```
int x = 7; if (x) {…}  // NO
```

  - Use relational operators: **if (x != 0)**
  - Avoids common mistakes, e.g. **if (x = 0)**

Control Statements

- Similar to C
  - if-else
  - switch
  - while
  - do-while
  - for
  - break
  - continue

Switch Statements with Strings

- Strings can be used as case values

```
switch (season) {
  case "summer":
  case "spring": temp = "hot";
                 break;
}
```

– The compiler generates more efficient bytecode from a switch on Strings than from chained if-then-else statements

Code Blocks and Scope

- Java code blocks are the same as in C
- Each block is enclosed by braces **{ }** and starts a new scope for the variables
- Variables can be declared both at the beginning and in the middle of a block

```
for (int i=0; i<10; i++) {
  int x = 12;
  ...
  int y;
  ...
}
```

Passing Parameters

- Parameters are always passed *by value*
- ...they can be primitive types or object *references*
  - Note: only the object reference is copied, not the whole object

Programming Errors

- Syntax Errors
  – Detected by the compiler
- Runtime Errors
  – Causes the program to abort
- Logic Errors
  – Produces incorrect result

Syntax Errors

```
public class ShowSyntaxErrors {
  public static main(String[] args) {
    System.out.println("Welcome to Java);
  }
}
```

▶ Try it live ↗

Runtime Errors

```
public class ShowRuntimeErrors {
  public static void main(String[] args) {
    System.out.println(1 / 0);
  }
}
```

▶ Try it live ↗

Logic Errors

```
public class ShowLogicErrors {
  public static void main(String[] args) {
    System.out.println("Celsius 35 is Fahrenheit degree ");
    System.out.println((9 / 5) * 35 + 32);
  }
}
```

$$T(°C) = (T(°F) - 32) / (9/5)$$

▶ Try it live ↗

https://evisp.github.io/java-oop-course/

Java OOP Course

Search

evisp/java-oop-course
☆11 2

Home Roadmap Tools Objects and State

Learn to think in objects

Fourteen weeks at University Metropolitan Tirana. You arrive able to write code that runs. You leave able to design scalable software that survives being changed.

Open the roadmap | Start week 1



Most programs start as a pile of requirements. The work is turning them into objects that own their own data.

One app, three sprints

You build it, rebuild it properly once you know what a weak design costs, then finish it. Not twelve exercises you throw away.

A portfolio, not a CV line

Your work is public on GitHub from week one. When somebody asks what you have built, you send them the repository.

Core skills

The ones you are asked to demonstrate after the course, not just describe.